

Pattern Class (java.util.regex.Pattern)

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>Pattern.compile(regex)</code>	Compiles the given regular expression string into a reusable Pattern object EXAMPLE: <code>Pattern p = Pattern.compile("\\d+");</code>
<code>Pattern.compile(regex, flags)</code>	Compiles pattern with one or more bitwise-OR flags (e.g. Pattern.CASE_INSENSITIVE) EXAMPLE: <code>Pattern p = Pattern.compile("abc", Pattern.CASE_INSENSITIVE);</code>
<code>Pattern.matches(regex, input)</code>	Quick one-liner check if regex matches the entire input CharSequence EXAMPLE: <code>Pattern.matches("\\d{4}", "2026") → true</code>
<code>pattern.matcher(input)</code>	Creates a Matcher object that will match the given input against this pattern EXAMPLE: <code>Matcher m = p.matcher(inputSequence);</code>
<code>pattern.split(input)</code>	Splits the input around matches of this pattern into a String[] array EXAMPLE: <code>p.split("apple,banana,cherry") → String[3]</code>
<code>pattern.split(input, limit)</code>	Splits input with threshold limit on the number of matching substrings EXAMPLE: <code>p.split("a:b:c:d", 2) → ["a", "b:c:d"]</code>
<code>pattern.splitAsStream(input)</code>	Creates a Stream<String> from the input split around matches (Java 8+) EXAMPLE: <code>p.splitAsStream(text).forEach(System.out::println);</code>
<code>pattern.asPredicate()</code>	Creates a Predicate<String> for filtering streams using matcher.find() (Java 8+) EXAMPLE: <code>list.stream().filter(p.asPredicate()).toList();</code>
<code>pattern.asMatchPredicate()</code>	Creates a Predicate<String> that matches entire string using matcher.matches() (Java 11+) EXAMPLE: <code>list.stream().filter(p.asMatchPredicate()).toList();</code>
<code>Pattern.quote(s)</code>	Returns a literal pattern String wrapped in \Q...\E for exact matching EXAMPLE: <code>Pattern.quote("user[1].id") → "\Quser[1].id\E"</code>
<code>pattern.pattern()</code>	Returns the original regular expression string from which this pattern was compiled EXAMPLE: <code>p.pattern() → "\\d+"</code>
<code>pattern.flags()</code>	Returns this pattern's match flags bitmask EXAMPLE: <code>p.flags() → 2 (Pattern.CASE_INSENSITIVE)</code>

Matcher Class (java.util.regex.Matcher)

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>m.find()</code>	Attempts to find the next subsequence of the input sequence that matches the pattern EXAMPLE: <code>while (m.find()) { ... }</code>
<code>m.find(start)</code>	Resets this matcher and attempts to find next match starting from the specified index EXAMPLE: <code>m.find(10) → boolean</code>
<code>m.matches()</code>	Attempts to match the entire input region against the pattern EXAMPLE: <code>m.matches() → true</code> for exact match
<code>m.lookingAt()</code>	Attempts to match the input sequence, starting at the beginning, against the pattern EXAMPLE: <code>m.lookingAt() → true</code> if prefix matches
<code>m.group()</code>	Returns the input subsequence matched by the previous match (group 0) EXAMPLE: <code>m.group() → "2026-08-19"</code>
<code>m.group(n)</code>	Returns the input subsequence captured by the given 1-based group index EXAMPLE: <code>m.group(1) → "2026"</code>
<code>m.group(name)</code>	Returns the input subsequence captured by the named group (?<name>...) (Java 7+) EXAMPLE: <code>m.group("year") → "2026"</code>
<code>m.groupCount()</code>	Returns the number of capturing groups in this matcher's pattern EXAMPLE: <code>m.groupCount() → 2</code>
<code>m.start()</code> / <code>m.end()</code>	Returns start (inclusive) and end (exclusive) character offset of the match EXAMPLE: <code>m.start() → 0, m.end() → 4</code>
<code>m.start(group)</code> / <code>m.end(group)</code>	Returns start / end offset of the subsequence captured by group index or name EXAMPLE: <code>m.start("year") → 0, m.end("year") → 4</code>
<code>m.replaceAll(replacement)</code>	Replaces every subsequence of input sequence that matches pattern with replacement EXAMPLE: <code>m.replaceAll("-") → "hello-world"</code>
<code>m.replaceFirst(replacement)</code>	Replaces first subsequence of input that matches pattern with replacement EXAMPLE: <code>m.replaceFirst("#") → "#123 456"</code>
<code>m.replaceAll(function)</code>	Replaces each match with result of applying function to MatchResult (Java 9+) EXAMPLE: <code>m.replaceAll(mr -> mr.group().toUpperCase())</code>
<code>m.results()</code>	Returns a Stream<MatchResult> of all match results in input order (Java 9+) EXAMPLE: <code>m.results().map(MatchResult::group).toList()</code>

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>m.appendReplacement(sb, repl)</code>	Non-terminal step: appends preceding text and replacement to StringBuffer/StringBuilder EXAMPLE: <code>m.appendReplacement(sb, newText);</code>
<code>m.appendTail(sb)</code>	Terminal step: appends remaining input text after last match to StringBuilder (Java 9+) EXAMPLE: <code>m.appendTail(sb);</code>
<code>m.reset([input])</code>	Resets this matcher with optional new input CharSequence EXAMPLE: <code>m.reset("new test string");</code>
<code>m.usePattern(newPattern)</code>	Changes the Pattern that this Matcher uses to find matches EXAMPLE: <code>m.usePattern(Pattern.compile("\\w+"));</code>

String Class Regex Methods

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>str.matches(regex)</code>	Returns true if and only if the entire string matches the given regular expression EXAMPLE: <code>"12345".matches("\\d+") → true</code>
<code>str.replaceAll(regex, repl)</code>	Replaces each substring matching regex with replacement string (supports \$1, \${name}) EXAMPLE: <code>"a1b2c3".replaceAll("\\d", "#") → "a#b#c#"</code>
<code>str.replaceFirst(regex, repl)</code>	Replaces first substring matching regex with replacement string EXAMPLE: <code>"foo 1 2".replaceFirst("\\d", "#") → "foo # 2"</code>
<code>str.split(regex)</code>	Splits string around matches of regex; trailing empty strings are discarded EXAMPLE: <code>"a,b,,c".split(",") → ["a", "b", "", "c"]</code>
<code>str.split(regex, limit)</code>	Splits string around matches with control on result array length (limit < 0 retains trailing empty strings) EXAMPLE: <code>"a:b:c:".split(":", -1) → ["a", "b", "c", ""]</code>

Java Pattern Flags

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>Pattern.CASE_INSENSITIVE</code>	Enables case-insensitive matching (Inline modifier: <code>(?i)</code> - ASCII by default; combine with <code>UNICODE_CASE</code> for full Unicode) EXAMPLE: <code>Pattern.compile("cat", Pattern.CASE_INSENSITIVE)</code>
<code>Pattern.MULTILINE</code>	<code>^</code> and <code>\$</code> match just after / before line terminators, as well as start / end of input (Inline modifier: <code>(?m)</code>) EXAMPLE: <code>Pattern.compile("^\\w+", Pattern.MULTILINE)</code>
<code>Pattern.DOTALL</code>	The dot <code>.</code> character matches any character including line terminators <code>(\n, \r)</code> (Inline modifier: <code>(?s)</code>) EXAMPLE: <code>Pattern.compile("<div>.*</div>", Pattern.DOTALL)</code>
<code>Pattern.COMMENTS</code>	Permits whitespace and comments starting with <code>#</code> inside pattern until end of line (Inline modifier: <code>(?x)</code>) EXAMPLE: <code>Pattern.compile("\\d{3} # area", Pattern.COMMENTS)</code>
<code>Pattern.UNICODE_CHARACTER_CLASS</code>	Enables Unicode version of predefined classes <code>(\w, \d, \s)</code> matching Unicode specs (Java 7+) (Inline modifier: <code>(?U)</code>) EXAMPLE: <code>Pattern.compile("\\w+", Pattern.UNICODE_CHARACTER_CLASS)</code>
<code>Pattern.UNICODE_CASE</code>	Enables Unicode-aware case folding when used alongside <code>Pattern.CASE_INSENSITIVE</code> (Inline modifier: <code>(?u)</code>) EXAMPLE: <code>Pattern.CASE_INSENSITIVE Pattern.UNICODE_CASE</code>
<code>Pattern.UNIX_LINES</code>	Only the <code>\n</code> character is recognized as a line terminator in <code>^</code> , <code>\$</code> , and <code>.</code> modes (Inline modifier: <code>(?d)</code>) EXAMPLE: <code>Pattern.compile("^line", Pattern.UNIX_LINES)</code>
<code>Pattern.CANON_EQ</code>	Enables canonical equivalence matching (e.g. character with accent vs combining accent) EXAMPLE: <code>Pattern.compile("â", Pattern.CANON_EQ)</code>
<code>Pattern.LITERAL</code>	Specifies that pattern contains literal characters with no metacharacter meanings EXAMPLE: <code>Pattern.compile(".*+?", Pattern.LITERAL)</code>
<code>Pattern.CASE_INSENSITIVE Pattern.MULTILINE</code>	Combine multiple flags using the bitwise OR <code>()</code> operator EXAMPLE: <code>Pattern.CASE_INSENSITIVE Pattern.MULTILINE</code>

Java-Specific Syntax, POSIX & Properties

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
"\\d", "\\b", "\\s"	Double backslash in Java strings: Java compiler uses 1st escape, regex engine uses 2nd EXAMPLE: <code>String pat = "\\b[A-Z]+\\b";</code>
"""\\d+"""	Java 15+ Text Blocks (triple quotes) simplify multiline regex, though backslashes still escape EXAMPLE: <code>"""\\d{4}-\\d{2}-\\d{2}"""</code>
(?<name>X)	Named capturing group — captures matching text accessible via <code>matcher.group("name")</code> EXAMPLE: <code>(?<zip>\\d{5}) → m.group("zip")</code>
\\k<name>	Named backreference within the regex pattern to an earlier named group EXAMPLE: <code>(?<quote>['"]).*?\\k<quote></code>
\${name} / \$1	Named and indexed group backreferences in <code>replaceAll()</code> replacement strings EXAMPLE: <code>m.replaceAll("Found: \${name} (id: \$1)")</code>
+	Possessive quantifier (also ++, ?+, {n,m}+) — eats greedily and never backtracks; prevents ReDoS vulnerabilities EXAMPLE: <code>"[^"]+" → matches quoted string fast</code>
[a-z&&[def]]	Character class intersection: matches characters present in both sets ('d', 'e', or 'f') EXAMPLE: <code>[a-z&&[aeiou]] → vowels only</code>
[a-z&&[^bc]]	Character class subtraction: matches a through z except 'b' and 'c' EXAMPLE: <code>[0-9&&[^2468]] → odd digits</code>
[a-d[m-p]]	Character class union: matches union of sets (equivalent to [a-dm-p]) EXAMPLE: <code>[a-c[1-3]] → [a-c1-3]</code>
\\p{Lower}, \\p{Upper}, \\p{Digit}	POSIX character classes: Lowercase [a-z], Uppercase [A-Z], Digits [0-9] EXAMPLE: <code>\\p{Digit}+ → matches numbers</code>
\\p{Alpha}, \\p{Alnum}, \\p{Punct}	POSIX alphabetic [a-zA-Z], alphanumeric, and punctuation characters EXAMPLE: <code>\\p{Alnum}+ → "User1042"</code>
\\p{Blank}, \\p{Space}, \\p{Cntrl}	POSIX space or tab [\\t], whitespace [\\t\\n\\x0B\\f\\r], and control character EXAMPLE: <code>\\p{Blank} → space or tab only</code>
\\p{javaLowerCase}, \\p{javaWhitespace}	Java character property classes matching <code>Character.isLowerCase()</code> / <code>isWhitespace()</code> EXAMPLE: <code>\\p{javaWhitespace}+</code>
\\p{IsGreek}, \\p{IsLatin}, \\p{Sc}	Unicode Scripts / Blocks and Categories (\\p{Sc} for currency symbols: \$, €, ¥) EXAMPLE: <code>\\p{Sc}\\d+ → "\$100", "€50"</code>

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\Q...\E</code>	Literal quotation — all characters between <code>\Q</code> and <code>\E</code> are treated as literal text EXAMPLE: <code>\Q*[a-z]\E</code> matches literal <code>"*[a-z]"</code>
<code>\R</code>	Any Unicode line break sequence (<code>\u000D\u000A \u000A ...</code>) (Java 8+) EXAMPLE: <code>Pattern.compile("\\R")</code> splits any OS newline
<code>\A, \Z, \z, \G</code>	Boundaries: <code>\A</code> (beginning of input), <code>\Z</code> (end except final terminator), <code>\z</code> (strict end of input), <code>\G</code> (end of previous match) EXAMPLE: <code>\\A[a-z]+\z</code>